IUT Montpellier-Sète
Département Informatique
M. Gasquet, G. Hisler, S. Robillard, P. Valicov

TD1 - Diagramme de classes de conception

Exercice 1 : Conception d'une classe

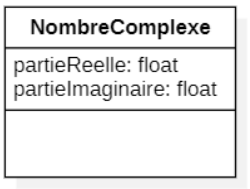
Dans le cadre de la réalisation d'un projet visant à produire une application de calculs, il a été émis le besoin de pouvoir gérer des **nombres complexes**.

Un **nombre complexe** est un nombre qui peut s'écrire sous la forme $a+bi$. Il est donc composé de deux parties :

- Une partie **réelle** : nombre réel (par exemple, 2 ou 5.36).
- Une partie **imaginaire** : nombre réel, multiplié par le nombre imaginaire i (défini par $i^2 = -1$).

Par exemple, $5 + 3i$ est un nombre complexe.

Un diagramme de classes d'analyse a été produit, contenant notamment la classe suivante :



Au début de la phase de conception, il a été déterminé qu'il doit être possible de :

- Instancier un nombre complexe sans aucune données (le nombre vaut alors $0 + 0i$).
- Instancier un nombre complexe à partir de sa partie réelle seulement (le nombre vaut alors $a + 0i$).
- Instancier un nombre complexe à partir de deux nombres réels (le nombre vaut alors $a + bi$).
- Instancier un nombre complexe en copiant les données d'un autre nombre complexe.
- Calculer, créer et retourner un nombre issu de l'addition du nombre complexe avec un autre nombre complexe.
- Calculer, créer et retourner un nombre issu du produit du nombre complexe avec un autre nombre complexe.
- Vérifier si le nombre complexe est "égal" à un autre objet.
- Afficher le nombre sous sa forme d'une chaîne de caractères $a + bi$.
- Lire les valeurs de la partie réelle et imaginaire du nombre, mais pas les modifier.
- Pouvoir connaître le nombre total de nombres complexes créés dans l'application.

1 - Donnez le **diagramme de classes de conception** contenant la classe **NombreComplexe** enrichie, notamment avec diverses méthodes. Posez-vous les bonnes question sur les types (des attributs, des paramètres, des valeurs de retour des méthodes...), les constructeurs, la visibilité de chaque élément, etc...

2 - Implémentez la classe **NombreComplexe** avec le langage **Java** (donnez son code complet).

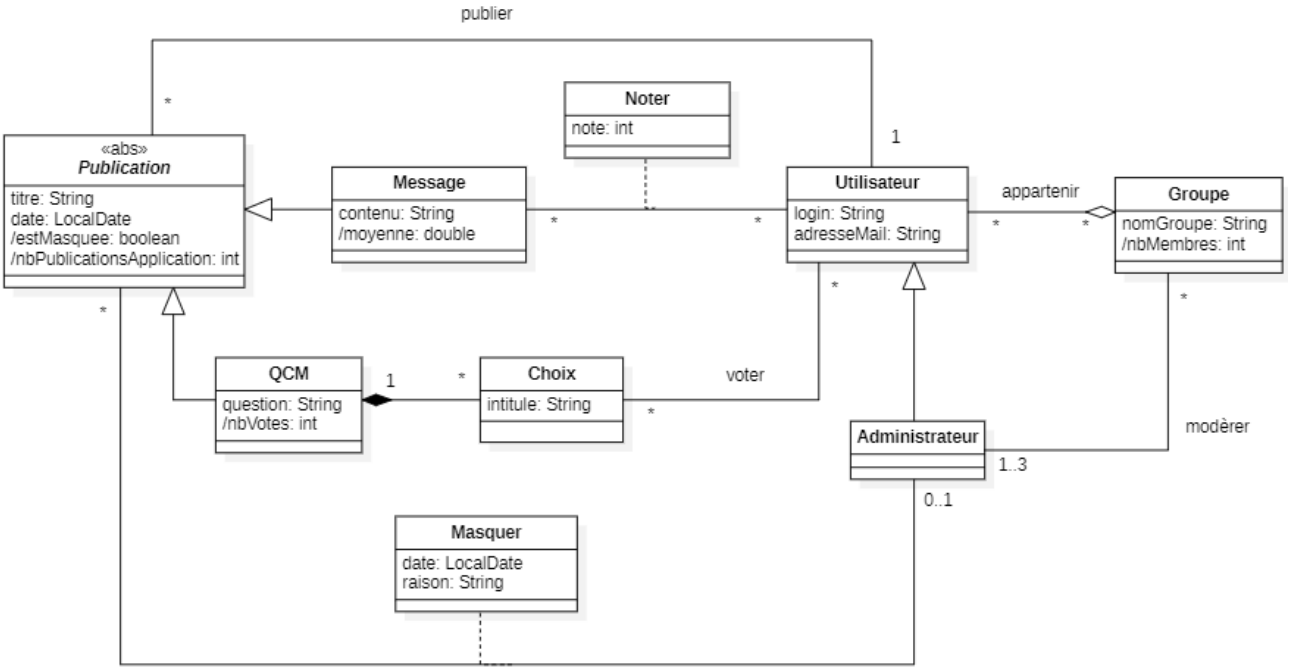
Au cas où les cours de mathématiques remontent un peu trop :

— $(a + bi) + (a' + b'i) = (a + a') + (b + b')i$

— $(a+bi)(a'+b'i) = aa'+ab'i+ba'i+bb'i^2 = aa'+ab'i+ba'i+(bb')(-1) = (aa'-bb')+(ab'+ba')i$

Exercice 2 : Conception plus poussée

On souhaite réaliser une application de gestion d'un mini réseau-social. La phase d'analyse a déjà eu lieu et l'équipe a produit le diagramme de classes d'analyse suivant :



On y retrouve toutes les **entités métier** qui nous serviront aussi à modéliser nos tables dans notre base de données.

On remarquera qu'il y a deux types de publications dans l'application : les messages et les QCM (sondages). Un message peut être noté par différents utilisateurs. Un QCM possède plusieurs choix (réponses) et un utilisateur peut voter pour plusieurs choix d'un même QCM. Toutes les publications peuvent être temporairement masquées par un administrateur.

Il existe différents groupes d'utilisateurs modérés par au moins un administrateur et au plus 3. Une publication (message ou QCM) peut être masquée par un administrateur, si besoin. On ne garde que les informations du masquage le plus récent (si la publication a été masquée).

Pour lancer la phase de conception, nous nous proposons d'utiliser ce diagramme comme base pour construire notre diagramme de classes de **conception**. En plus de cet outil, voici quelques pistes de réflexion de l'équipe du projet, par rapport à la conception à adopter :

Concernant les **utilisateurs** :

- On peut récupérer le login et l'adresse mail d'un utilisateur.
- On peut seulement modifier l'adresse mail d'un utilisateur.
- On peut récupérer les groupes d'un utilisateur, en ajouter, en supprimer.

Concernant les **groupes** :

- On peut récupérer le nom d'un groupe et son nombre de membres.
- On peut récupérer les membres d'un groupe, en ajouter, en supprimer.
- On peut récupérer les modérateurs d'un groupe, en ajouter, en supprimer.
- On peut seulement modifier le nom d'un groupe.

Concernant les **publications** :

- On peut récupérer le titre, la date, l'auteur d'une publication et le nombre total de publications créées dans l'application.
- On peut seulement modifier le titre d'une publication.
- Quand une publication est créée, sa date de publication est générée automatiquement.
- On peut masquer une publication par un appel de méthode, en précisant la raison et l'administrateur qui effectue l'action. La date de masquage est générée automatiquement. Les informations de masquage doivent être détruites si la publication est détruite.
- On ne peut pas modifier les informations de masquage d'une publication.
- On peut vérifier qu'une publication est masquée ou non et aussi la démasquer.
- Si une publication est masquée, on peut récupérer une chaîne de caractères qui détaille la raison du masquage, la date et l'administrateur ayant effectué cette action.

Concernant les **messages** :

- On peut récupérer le contenu d'un message et sa moyenne (la moyenne de ses notes).
- On peut seulement modifier le contenu d'un message.
- On peut ajouter une note à un message en précisant l'auteur de la note et sa valeur. Le message se chargera de créer et de stocker la note à partir de ces informations.
- Il n'y a pas d'intérêt à garder les notes d'un message si le message en question est supprimé.
- On peut récupérer le détail des différentes notes d'un message sous la forme d'un tableau de chaînes de caractères avec, pour chaque entrée, une chaîne avec le nom de l'utilisateur et la note qu'il a donné.

Concernant les **QCM** :

- On peut récupérer la question d'un QCM et le nombre d'utilisateurs total ayant voté.
- On peut ajouter des choix à un QCM, en précisant l'intitulé du choix.
- On peut supprimer un choix d'un QCM en indiquant l'index du choix.
- On peut ajouter ou supprimer une vote pour un choix d'un QCM en précisant l"index du choix et l'utilisateur votant.
- On peut récupérer le détail des différents votes du QCM sous la forme d'un tableau de chaînes de caractères avec, pour chaque entrée, une chaîne avec l'intitulé du choix et le nombre de votes.

1 - A partir du diagramme de classe d'analyse, des pistes de réflexions de l'équipe projet, de vos connaissances et de votre interprétation du problème, proposez une première ébauche du **diagramme de classes de conception** de cette application, incluant les **entités métiers** présentées dans le premier diagramme. Réfléchissez bien aux **associations**, à leur **navigabilité** (unidirectionnel, bi-directionnel ?), aux méthodes à ajouter, au(x) constructeur(s), à la visibilité des attributs...

Ce premier diagramme ne contient que les entités **métiers**, soit une toute petite partie de l'application ! L'application va évoluer et beaucoup de classes non-persistantes vont être ajoutées au fur et à mesure de la phase de conception et de l'implémentation.

Exercice 3 : Du code au diagramme (rétro-ingénierie)

On considère le code des classes suivantes, représentant l'implémentation d'une petite application de jeu vidéo type RPG :

```java
public interface Entite {
    Item volerItem();
}
```

```java
public class Joueur implements Entite {

    private String nom;
    private int argent = 0;
    private float pvMax = 1000.0f;
    private float pv;
    private Inventaire sac;
    private Session session;

    public Joueur(String nom) {
        this.nom = nom;
        pv = pvMax;
        sac = new Inventaire(this);
    }

    public void retirerArgent(int valeur) {
        argent -= valeur;
        if(argent < 0) {
            argent = 0;
        }
    }

    public void subirDegats(float degats) {
        pv -= degats;
        if(pv <= 0) {
            mourir();
        }
    }

    public void voler(Entite entite) {
        Item item = entite.volerItem();
        if(item != null) {sac.ajouterItem(item);}
    }
```

```
    public Item volerItem() {
        if(!sac.estVide()) {
            return sac.supprimerItem(0);
        }
        return null;
    }

    public String getNom() {return nom;}

    public void setNom(String nom) {this.nom = nom;}

    public Inventaire getSac() {return sac;}

    public void ajouterArgent(int valeur) {argent += valeur;}

    public void soigner(float nbPvSoin) {pv = Math.max(pvMax, pv+nbPvSoin);}

    public void mourir() {session.incrementerMorts();}

    public void connexion() {session = new Session();}

    public void deconnexion() {session = null;}

    public int getNbMortsSession() {return session.getNbMorts();}

    public int getTempsEnJeu() {return session.getDureeSession();}
}
```

```
public class Session {

    private LocalDate dateHeureConnexion;
    private int nbMorts;

    public Session() {
        nbMorts = 0;
        dateHeureConnexion = LocalDate.now();
    }

    public int getNbMorts() {return nbMorts;}

    public void incrementerMorts() {nbMorts++;}

    public LocalDate getDateHeureConnexion() {return dateHeureConnexion;}

    public int getDureeSession() {
        LocalDate maintenant = LocalDate.now();
        return (int) ChronoUnit.MINUTES.between(dateHeureConnexion, maintenant);
    }
}
```

```java
public class Inventaire {

    private Joueur proprietaire;
    private List<Item> items = new ArrayList<>();

    public Inventaire(Joueur proprietaire) {this.proprietaire = proprietaire;}

    public Joueur getProprietaire() {return proprietaire;}

    public void ajouterItem(Item item) {items.add(item);}

    public Item supprimerItem(int position) {return items.remove(position);}

    public float getPrixTotalInventaire() {
        float prix = 0.0f;
        for(Item item : items) {
            prix += item.getPrix();
        }
        return prix;
    }

    public boolean estVide() {return items.isEmpty();}
}
```

```java
public class Monstre implements Entite {

    private int id;

    private String nom;

    private float force;

    private Item item;

    public Monstre(int id, String nom, float force, Item item) {
        this.id = id;
        this.nom = nom;
        this.force = force;
        this.item = item;
    }

    public Item volerItem() {
        Item itemVole = item;
        item = null;
        return itemVole;
    }

    public void attaquerCible(Joueur joueur) {
        joueur.subirDegats(force);
    }
}
```

```java
public enum Rarete {

    COMMUN(1),
    RARE(2),
    LEGENDAIRE(3);

    public final int niveau;

    Rarete(int niveau) {
        this.niveau = niveau;
    }

}
```

```java
public abstract class Item {

    private static int nbItemsEnJeu = 0;
    private String nom;
    private final Rarete rarete;
    private Inventaire inventaire;

    public Item(String nom, Rarete rarete) {
        nbItemsEnJeu++;
        this.nom = nom;
        this.rarete = rarete;
    }

    public String getNom() {return nom;}

    public Rarete getRarete() {return rarete;}

    public static int getNbItemsEnJeu() {return nbItemsEnJeu;}

    public abstract float getValeur();

    public float getPrix() {return rarete.niveau * getValeur();}

    public boolean estDansUnInventaire() {return inventaire != null;}

    public void deplacer(Inventaire inventaire) {
        this.inventaire = inventaire;
        this.inventaire.ajouterItem(this);
    }

    public String getNomProprietaire() {return inventaire.getProprietaire().getNom();}
}
```

```java
public interface Coffre {
    void ouvrir(Joueur joueur);
}
```

```java
public class CoffrePotion implements Coffre {
    public void ouvrir(Joueur joueur) {
        float nbPv = (new Random().nextFloat() * 50.0f) + 100.0f;
        Potion p = new Potion("Potion de vie normale", Rarete.COMMUN, nbPv, 5);
        joueur.getSac().ajouterItem(p);
    }
}
```

```java
public class Potion extends Item {

    private final float nbPvRendus;
    private int utilisationsRestantes;

    public Potion(String nom, Rarete rarete, float nbPvRendus, int nbUtilisationsMax) {
        super(nom, rarete);
        this.nbPvRendus = nbPvRendus;
        utilisationsRestantes = nbUtilisationsMax;
    }

    public float getValeur() {return (nbPvRendus * 0.3f) * utilisationsRestantes;}

    public int getUtilisationsRestantes() {return utilisationsRestantes;}

    public float getNbPvRendus() {return nbPvRendus;}

    private void diminuerUtilisationsRestantes() {utilisationsRestantes--;}

    public void consommer(Joueur joueur) {
        joueur.soigner(getNbPvRendus());
        diminuerUtilisationsRestantes();
    }
}
```

```java
public class Arme extends Item {

    private float degats;

    public Arme(String nom, Rarete rarete, float degats) {
        super(nom, rarete);
        this.degats = degats;
    }

    public float getValeur() {return degats * 0.50f;}

    public void taper() {
        System.out.printf("Le joueur frappe et inflige %s degats\n", degats);
    }

    public float getDegats() {return degats;}

    public void setDegats(float degats) {this.degats = degats;}
}
```

```java
public enum TypeSort {
    FOUDRE(15), FEU(20), GLACE(10);

    public float degatsMagique;

    Sort(float degatsMagiques) {this.degatsMagique = degatsMagiques;}
}
```

```java
public class Sort {
    private TypeSort type;
    private float puissance;

    public Sort(TypeSort type, float puissance) {
        this.type = type;
        this.puissance = puissance;
    }

    public float calculerDegats() {return type.degatsMagique + puissance;}

    public Sort copier() {return new Sort(type, puissance);}
}
```

```java
public class BatonSort extends Arme {

    private Sort sort;

    public BatonSort(String nom, Rarete rarete, float degats) {
        super(nom, rarete, degats);
    }

    public void enchanter(TypeSort type, float puissance) {sort = new Sort(type, puissance);}

    public void desanchanter() {sort = null;}

    public Sort getSort() {return sort != null ? sort.copier() : null;}

    public float getValeur() {return super.getValeur() * (getDegatsMagiquesTotal() * 0.15f);}

    private float getDegatsMagiquesTotal() {
        float degatsMagiques = 0.0f;
        if(sort != null) {
            degatsMagiques = sort.calculerDegats();
        }
        return degatsMagiques;
    }
}
```

1 - Donnez le **diagramme de classes de conception** de cette application (sur papier ou avec un logiciel de dessin, sans utiliser d'IDE).

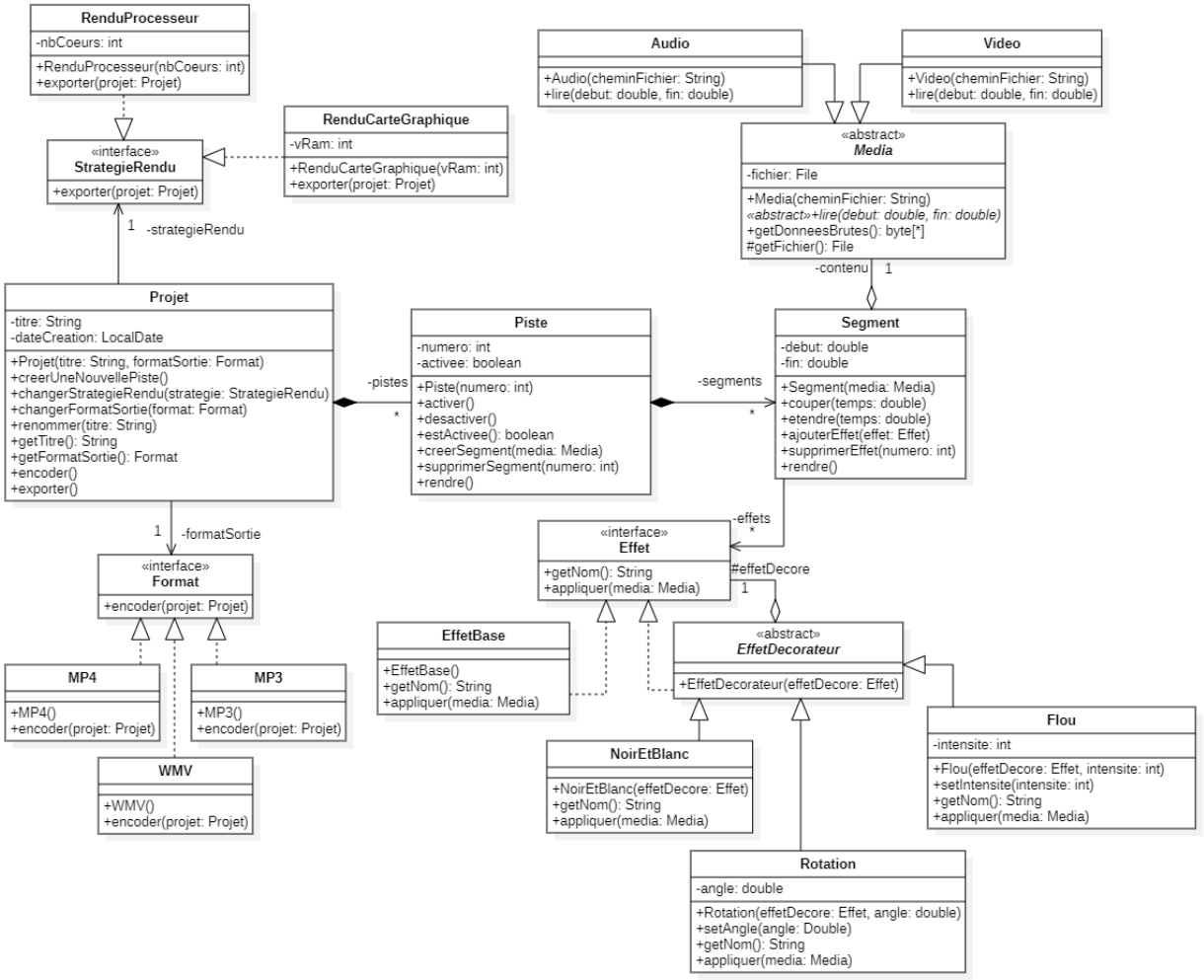
2 - Ce code contient plusieurs **dépendances non-triviales**. Est-ce que vous les avez trouvées ? Si ce n'est pas le cas, cherchez-les puis ajoutez-les sur votre diagramme.

3 - Peut-on refactorer notre application pour éliminer certaines de ces dépendances ?

4 - Sur machine, créez un nouveau projet Java avec **IntelliJ** et copiez/collez le code de toutes les classes dans les fichiers adéquats. Générez ensuite le **diagramme de classes** afin de comparer avec celui que vous avez dessiné. Pour cela, sélectionnez votre projet, faites un clic droit puis sélectionnez **Diagrams** → **Show Diagrams** → **Java Classes**. Activez les diverses options (boutons en haut du diagramme) pour voir les champs, les dépendances, etc.

Exercice 4 : Du diagramme au code

On considère le diagramme de classes suivant :



Ce diagramme de classes de conception représente une application de montage (vidéo, audio).

1 - Quelles **dépendances** non triviales pourrait-on probablement déjà ajouter sur ce diagramme (avant même d'implémenter cette conception) ?

2 - Sur machine, implémentez le **squelette de code** (en Java, avec IntelliJ) correspondant à l'implémentation de cette conception. Par "squelette" on entend qu'on doit avoir le code de la déclaration de chaque classe, de ses attributs, et la signature de ses méthodes (sans implémenter le code contenu dans ces méthodes). Pour les méthodes qui doivent retourner quelque-chose, vous pourrez simplement retourner **null** ou une valeur par défaut.

3 - Une fois que le squelette de code est achevé, générez le **diagramme de classes** avec IntelliJ afin de comparer avec celui donné dans cet exercice (et ainsi vérifier votre solution).